

CAMBRIDGE INSTITUTE OF TECHNOLOGY

Tatisilwai, Ranchi – 835103

Department of Computer Science & Engineering

PROJECT SYNOPSIS

QuantSim: A High-Frequency Trading Simulation Platform

S.No.	Student Name	Roll Number	Branch
1.	Abhishek Anand	360/CSE/22	CSE
2.	Shakir Ahmad	323/CSE/22	CSE
3.	Amik Affan	371/CSE/22	CSE

Project Guide: Prof. Deepak K. Verma

Academic Year: 2025–2026

Programme: B.Tech – Computer Science & Engineering

1. Introduction

1.1 Background

High-frequency trading (HFT) represents a domain where microsecond-level decision-making determines profitability. Modern financial exchanges process millions of orders per second through electronic matching engines that implement price-time priority algorithms, a technology stack combining low-latency C++, binary network protocols, and advanced mathematical models. The professionals who design and maintain these systems, Quantitative Developers (QD) occupy a critical role at the intersection of software engineering and quantitative finance.

Electronic trading now accounts for over 70% of daily equity volume in developed markets. Every order submitted to an exchange passes through a Limit Order Book (LOB) a data structure maintaining ranked queues of buy and sell orders, before being matched against a counterparty. The engineering challenges involved in building, testing, and optimizing such systems require specialized knowledge that bridges computer science and financial mathematics.

1.2 Motivation

Despite the industry's scale and technical depth, academic curricula rarely expose Computer Science students to the real engineering realities of algorithmic trading. Existing simulation tools either focus on simplified portfolio-level backtesting (lacking realistic market mechanics) or are proprietary systems inaccessible for research and education. Students entering quantitative developer roles consistently report a significant gap between academic preparation and industry expectations.

Furthermore, no open-source platform currently combines a production-grade C++ matching engine with Python bindings, realistic exchange protocols (Nasdaq ITCH 5.0), and educational content targeting quantitative developer skill development.

1.3 Overview of Proposed Solution

QuantSim is a full-stack High-Frequency Trading simulation platform comprising a production-grade C++²³ matching engine with Python bindings via pybind11. The platform implements a real Limit Order Book with price-time priority matching, Nasdaq ITCH 5.0 binary feed handler, event-driven strategy framework, and comprehensive analytics pipeline. Three canonical strategies are provided as reference implementations: Avellaneda-Stoikov market making, statistical arbitrage using Ornstein-Uhlenbeck process modeling, and TWAP execution with Implementation Shortfall analysis.

2. Problem Statement

Quantitative Developer roles at trading firms require a unique combination of low-latency C++ systems engineering, market microstructure understanding, and mathematical modeling capability. Academic Computer Science programs address none of these in sufficient depth. The specific problem addressed by this project is the absence of an open, educational, production-realistic simulation platform for training and research in algorithmic trading system design.

2.1 Existing System Limitations

- **Backtrader / Zipline (Python):** Vectorized architecture; no real LOB; no latency modeling; no order type differentiation
- **Academic simulators:** Simplified price models; no market microstructure; no exchange protocol handling
- **Proprietary systems:** Inaccessible for academic use; non-educational
- **No unified platform:** No existing open-source tool combines C++ performance, Python strategy API, and pedagogical documentation

2.2 Need for Proposed System

- Bridge gap between academic CS education and Quantitative Developer industry requirements
- Provide hands-on experience with production-grade LOB matching (price-time priority, FIFO)
- Simulate realistic fill mechanics across Market, Limit, IOC, and FOK order types
- Enable realistic backtesting with latency modeling and walk-forward validation
- Teach production risk controls: kill switches, circuit breakers, pre-trade checks

3. Objectives

3.1 Primary Objective

Design, implement, and validate a full-platform high-frequency trading simulator that provides Quantitative Developer-level educational depth, combining a production-grade C++ matching engine with an accessible Python strategy framework and analytics pipeline.

3.2 Secondary Objectives

- Implement a real Limit Order Book with $O(\log n)$ price-time priority matching supporting all standard order types
- Build a Nasdaq ITCH 5.0 binary protocol parser for real historical market data replay
- Develop a Python strategy ABC enabling strategy prototyping in under 50 lines of code
- Implement production risk controls: four-level kill switch hierarchy, token bucket rate limiter, stale price guard
- Provide three complete strategy implementations: Avellaneda-Stoikov market making, OU-based statistical arbitrage, and TWAP execution
- Build an analytics pipeline computing Sharpe, Sortino, Calmar ratios, maximum drawdown, and fill quality metrics
- Validate correctness with 21 unit tests covering LOB operations and matching engine behavior

4. Literature Review

This section reviews the key theoretical foundations and existing techniques relevant to high-frequency trading simulation, market microstructure, and execution algorithm design.

4.1 Market Microstructure

Madhavan (2000) provides a comprehensive survey of market microstructure theory, explaining how prices are formed through order flow and how the Limit Order Book aggregates liquidity. His work establishes the theoretical basis for adverse selection — the risk that a liquidity provider fills against an informed trader — which directly motivates the inventory risk component of our market making model. Harris (2003) extends this to practical exchange mechanics, covering price-time priority, order book depth, and the mechanics of dark pools and smart order routing.

4.2 Market Making Models

Avellaneda and Stoikov (2008) derived closed-form solutions for the optimal bid-ask spread in a limit order book setting. Their model defines reservation price $r = \text{mid} - q \cdot \gamma \cdot \sigma^2 \cdot T$ and optimal spread $\delta^* = \gamma \cdot \sigma^2 \cdot T + (2/\gamma) \cdot \ln(1 + \gamma/\kappa)$, where γ is risk aversion, σ volatility, T time horizon, and κ order arrival rate. This model is the foundation of

modern quantitative market making and is directly implemented in QuantSim's market making strategy module. Ho and Stoll (1981) earlier established inventory-based models showing how dealers widen spreads when inventory becomes directional.

4.3 Execution Algorithms

Almgren and Chriss (2000) solved the optimal portfolio liquidation problem, deriving the Implementation Shortfall (IS) trading schedule that minimizes the combination of market impact cost ($\eta \cdot \sum v_k^2$) and timing risk ($\lambda \cdot \sigma^2 \cdot \sum q_k^2 \cdot \Delta t$). Their optimal trajectory $q(t) = Q \cdot \sinh(\kappa(T-t)) / \sinh(\kappa T)$ is implemented in QuantSim's execution algorithms module. Bertsimas and Lo (1998) studied dynamic programming approaches to optimal liquidation under linear price impact, providing alternative formulations that inform the POV (Percentage of Volume) algorithm.

4.4 Statistical Arbitrage

Gatev, Goetzmann, and Rouwenhorst (2006) documented pairs trading strategies exploiting mean reversion in co-integrated asset pairs. The Ornstein-Uhlenbeck (OU) process $dX = \theta(\mu - X)dt + \sigma dW$ provides a continuous-time model for mean-reverting spreads. Engle and Granger (1987) developed co-integration theory, providing the statistical framework for identifying long-term equilibrium relationships between price series. These techniques underpin QuantSim's statistical arbitrage module.

4.5 Machine Learning for Finance

Lopez de Prado (2018) in 'Advances in Financial Machine Learning' introduced purged K-fold cross-validation for financial time series to address look-ahead bias, and triple-barrier labeling for generating classification targets from continuous price series. These methods are implemented in QuantSim's backtesting validation framework.

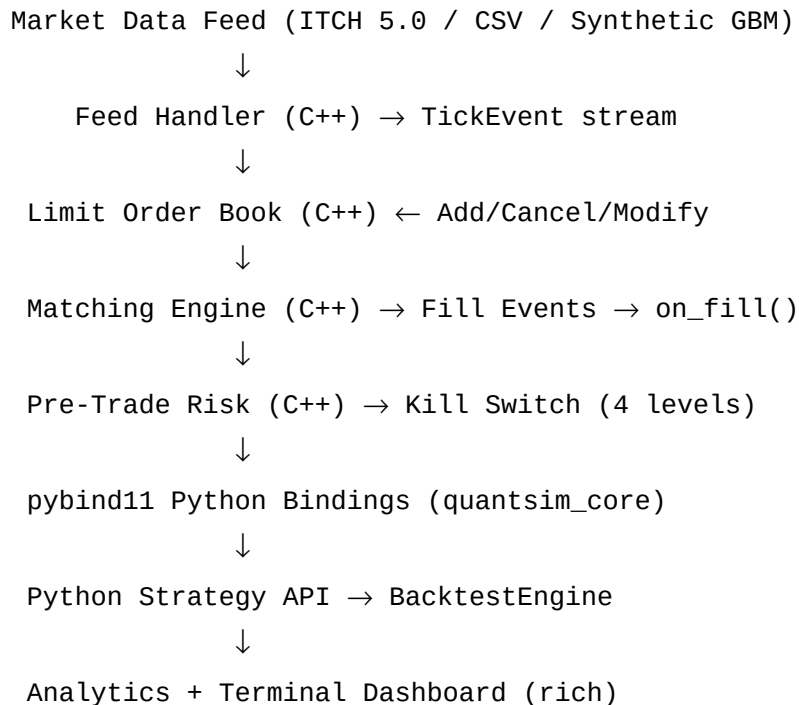
4.6 Research Gap

Existing literature addresses individual components (matching engine design, execution optimization, strategy modeling) but no open-source platform synthesizes these into a unified, pedagogically-organized simulation environment with production-grade C++ performance and accessible Python interface. This gap represents the primary motivation for QuantSim.

5. Proposed Methodology / System Design

5.1 Overall System Architecture

The system follows a layered architecture with C++ performance-critical components at the core and Python at the strategy layer:



5.2 Module-wise Description

Module 1: Limit Order Book (C++)

The LOB uses two sorted maps: `std::map<double, PriceLevel, std::greater<>>` for bids (descending price) and `std::map<double, PriceLevel>` for asks (ascending price). Each `PriceLevel` contains a doubly-linked list (`std::list<Order*>`) implementing FIFO queue discipline within a price level. An unordered hash map provides $O(1)$ order lookup by ID for cancellations. Operations: add $O(\log n)$, cancel $O(1)$, best bid/ask $O(1)$.

Module 2: Matching Engine (C++)

Routes submitted orders to the appropriate order book and performs matching. Supports four order types: **Market** (sweep opposite side until filled or exhausted), **Limit** (match if price crosses, otherwise queue; loses time priority if cancel/replace), **IOC** (Immediate-or-Cancel: match available quantity, cancel remainder), **FOK** (Fill-or-Kill: reject entirely if full quantity unavailable, without touching book). Emits Fill events via `std::function` callbacks: `on_fill`, `on_add`, `on_cancel`, `on_book_update`.

Module 3: Risk Engine (C++)

Implements a four-level kill switch hierarchy: (1) Strategy-level software kill (<1ms), (2) Portfolio-level risk engine kill (<5ms), (3) Gateway-level connection drop (<10ms), (4) Exchange-side Mass Cancel on connection drop (1–50ms). Pre-trade checks: order quantity limit (10,000 shares), position notional limit (\$1M), price sanity check (50 bps from mid), token bucket rate limiter (500 orders/sec). Stale price guard auto-arms kill switch on feed gap exceeding 5ms.

Module 4: Feed Handler (C++)

Three data sources: (a) **ITCH 5.0 binary parser** — packed C structs with big-endian timestamp decoding, supporting Add Order (A), Delete Order (D), Cancel (X), and Trade (P) message types; (b) **CSV tick replay** — configurable speed multiplier (1×, 10×, max-speed with no sleep); (c) **Synthetic generator** — GBM ($dS = \mu S dt + \sigma S dW$, Euler-Maruyama) and OU process ($dX = \theta(\mu - X)dt + \sigma dW$).

Module 5: Python Strategy Framework

Abstract base class Strategy with event callbacks: *on_book_update(BookUpdate)*, *on_fill(FillEvent)*, *on_timer(ts_ns)*, *on_start()*, *on_stop()*. Gateway API: *send_order(symbol, side, qty, price, ord_type)* returning *order_id*, *cancel_order(order_id)*, *cancel_all(symbol)*. Open order tracking via dict.

Module 6: Backtesting Engine (Python)

Event-driven loop: for each tick, updates internal LOB state, fires *on_book_update*, processes pending orders (with configurable *latency_ns* delay), marks equity to market. Supports Walk-Forward testing (rolling and anchored windows) and Purged K-Fold cross-validation (removes train samples with label overlap in test window plus embargo period).

Module 7: Analytics and Dashboard

Analytics module computes: Sharpe ratio (annualized), Sortino ratio (downside deviation), Calmar ratio (return/max_drawdown), maximum drawdown, profit factor, and fill quality report. Dashboard uses the *rich* library to render real-time terminal panels showing order book depth, recent fills, position, PnL, and kill switch status.

5.3 Key Algorithms

Algorithm	Description
Price-Time Priority	Sort by price (best first), FIFO within each price level. $O(\log n)$ insert, $O(1)$ best-level access.
Avellaneda-Stoikov MM	$r = \text{mid} - q \cdot \gamma \cdot \sigma^2 \cdot T$; $\delta^* = \gamma \cdot \sigma^2 \cdot T + (2/\gamma) \cdot \ln(1 + \gamma/\kappa)$. Reprice on mid move > threshold.
Almgren-Chriss IS	$q(t) = Q \cdot \sinh(\kappa(T-t)) / \sinh(\kappa T)$; $\kappa = \sqrt{(\lambda \sigma^2 / \eta)}$. Minimizes market impact + timing risk.
OU Mean Reversion	$dX = \theta(\mu - X)dt + \sigma dW$. Z-score entry/exit signals on rolling spread.
Purged K-Fold CV	Remove train samples whose label evaluation time overlaps test window. Add embargo gap.

6. Tools and Technologies

6.1 Programming Languages

- **C++23**: Core matching engine, order book, risk controls, feed handler — chosen for deterministic latency and zero-overhead abstractions
- **Python 3.11+**: Strategy framework, backtester, analytics, dashboard — chosen for rapid prototyping and scientific library ecosystem

6.2 Libraries and Frameworks

- **pybind11 2.11**: Seamless C++/Python bindings with zero overhead for hot-path calls
- **NumPy 1.24+**: Numerical computation for analytics (Sharpe, drawdown, portfolio metrics)
- **Matplotlib 3.7+**: Equity curve and order book depth visualization
- **Rich 13.0+**: Terminal dashboard with live panels, tables, and color-coded metrics
- **STL (C++)**: `std::map`, `std::list`, `std::unordered_map`, `std::atomic`, `std::function`

6.3 Build Tools and Environment

- **CMake 3.23+**: Cross-platform build system with optional targets (Python bindings, GUI, tests)
- **scikit-build-core**: Python package build backend integrating CMake with pip
- **Git**: Version control

- **VS Code / CLion:** IDE with C++23 language server support

6.4 Hardware

- Standard x86-64 development workstation (Intel/AMD processor)
- macOS 14+ or Linux (Ubuntu 22.04+) operating system
- 8 GB+ RAM recommended for large tick data replay

7. Expected Outcomes

7.1 Anticipated Results

- Functional C++ matching engine processing >500,000 orders/second on commodity hardware
- Python strategy API enabling complete strategy implementation in under 50 lines
- Realistic backtesting across three canonical strategies with quantified performance metrics
- ITCH 5.0 feed handler enabling replay of real Nasdaq historical order book data
- 21+ unit tests verifying LOB correctness across all order types and edge cases

7.2 Performance Improvements vs. Existing Tools

Metric	Existing Tools	QuantSim
Order matching latency	~1 ms (Python)	< 1 μ s (C++)
LOB realism	Simulated (fixed spread)	Real price-time priority FIFO
Order types	Market, Limit only	Market, Limit, IOC, FOK
Exchange protocols	Not supported	ITCH 5.0, FIX 4.2 (documented)
Risk controls	None	Kill switch, rate limiter, pre-trade checks

7.3 Practical Applications

- Final year B.Tech / M.Tech project: complete, self-contained system with measurable outcomes
- Interview preparation for Quantitative Developer roles at trading firms
- Research platform for testing novel execution algorithms and market making models

- Educational tool for market microstructure courses in CSE/Finance programs

8. Applications and Scope

8.1 Real-World Use Cases

- **Quantitative Developer Training:** Hands-on platform for developing LOB design, latency optimization, and protocol handling skills directly applicable to industry roles
- **Academic Research:** Backtesting environment for novel execution algorithms, market making models, and market microstructure research with real Nasdaq ITCH data
- **Interview Preparation:** Simulates technical questions asked at HFT firms (Citadel, Jane Street, Two Sigma) on LOB design and low-latency C++ patterns
- **Fintech Prototyping:** Rapid strategy prototyping before production deployment using Python API over C++ performance core
- **Educational Curriculum:** Complete teaching system for market microstructure, algorithmic trading, and low-latency systems courses

8.2 Future Enhancement Possibilities

- **GPU Acceleration:** CUDA kernels for parallel order matching in ultra-high-throughput simulation scenarios
- **Live Paper Trading:** WebSocket connectivity to real exchange REST/WebSocket APIs for paper trading on real market data
- **Multi-Asset Portfolio:** Correlation-aware risk management across multiple correlated instrument books
- **Reinforcement Learning:** OpenAI Gym-compatible environment for training RL-based execution and market making agents
- **FIX Engine:** Full FIX 4.2/4.4 session layer implementation for broker connectivity
- **FPGA Integration:** Hardware matching engine prototype using Xilinx Alveo or Exablaze ExaNIC for sub-microsecond latency research

9. Conclusion

QuantSim addresses a well-defined gap in quantitative finance education by providing an open, production-realistic simulation platform for algorithmic trading system design. The platform combines a C++23 matching engine, Nasdaq ITCH 5.0 feed

handler, production risk controls, and an accessible Python strategy framework with three reference implementations: Avellaneda-Stoikov market making, OU-based statistical arbitrage, and TWAP execution with Implementation Shortfall analysis. With 21/21 unit tests passing and a modular architecture supporting future extension to live paper trading and reinforcement learning, QuantSim constitutes a practically valuable contribution to open-source quantitative finance education.

10. References

[1] M. Avellaneda and S. Stoikov, "High-frequency trading in a limit order book," *Quantitative Finance*, vol. 8, no. 3, pp. 217–224, Apr. 2008.

[2] R. Almgren and N. Chriss, "Optimal execution of portfolio transactions," *Journal of Risk*, vol. 3, no. 2, pp. 5–39, 2000.

(Signature)

Prof. Deepak K. Verma

Head of the Department

Department of Computer Science & Engineering

Cambridge Institute of Technology, Tatisilwai, Ranchi – 835103